

Lekcja 8

Temat: Funkcje zaawansowane: Lambdy, map, filter, reduce; dekoratory

Funkcje lambda

Funkcje lambda **to anonimowe funkcje**, które można definiować w jednej linii. Przydatne do tworzenia prostych funkcji na potrzeby jednorazowego użycia, np. W **połączeniu z innymi funkcjami jak map, filter czy sorted**. Nie mają nazwy i składają się z słowa kluczowego lambda, argumentów, dwukropka i wyrażenia.

Przykład zastosowania: Sortowanie listy słowników po wartości klucza. Załóżmy, że mamy listę osób i chcemy posortować je po wieku.

```
osoby = [{'imie': 'Anna', 'wiek': 25}, {'imie': 'Jan', 'wiek': 30}, {'imie': 'Maria',  
'wiek': 22}]  
posortowane = sorted(osoby, key=lambda x: x['wiek'])  
print(posortowane)
```

Przykład: Obliczenie kwadratów liczb w liście.

```
kwadraty = list(map(lambda x: x**2, [1, 2, 3, 4]))  
print(kwadraty)
```

Funkcja map

Funkcja **map stosuje podaną funkcję do każdego elementu iterowalnego** (np. listy, tupli) i **zwraca iterator z wynikami**. Często łączy się ją z lambda dla zwięzłości.

Przykład zastosowania: Podwojenie elementów listy.

```
liczby = [1, 2, 3, 4]  
podwojone = list(map(lambda x: x * 2, liczby))  
print(podwojone)
```

Przykład: Konwersja temperatur z Celsjusza na Fahrenheita dla listy wartości.

Konwersja z **Celsjusza (°C)** na **Fahrenheita (°F)** działa według prostego wzoru matematycznego:

$$^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$$

Skale różnią się w dwóch rzeczach:

1. Wielkość stopnia

- a. $100^{\circ}\text{C} = 180^{\circ}\text{F}$
 $\rightarrow 1^{\circ}\text{C} = 9/5^{\circ}\text{F}$ (czyli 1.8°F)

2. Punkt zerowy

- a. $0^{\circ}\text{C} = 32^{\circ}\text{F}$
 $\rightarrow$ dlatego dodajemy **+32**

Przykłady

$0^{\circ}\text{C} \rightarrow (0 \times 9/5) + 32 = 32^{\circ}\text{F}$

$25^{\circ}\text{C} \rightarrow (25 \times 9/5) + 32 = 77^{\circ}\text{F}$

$100^{\circ}\text{C} \rightarrow (100 \times 9/5) + 32 = 212^{\circ}\text{F}$

```
celsjusze = [0, 10, 20, 30]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsjusze))
print(fahrenheit)
```

Funkcja filter

Funkcja **filter** filtruje elementy iterowalnego na podstawie warunku podanego w funkcji (zwracającej True/False). Zwraca iterator z elementami spełniającymi warunek.

Przykład zastosowania: Wybór parzystych liczb z listy.

```
liczby = [1, 2, 3, 4, 5, 6]
parzyste = list(filter(lambda x: x % 2 == 0, liczby))
print(parzyste)
```

Przykład: Filtrowanie słów dłuższych niż 3 litery z listy.

```
slova = ['kot', 'pies', 'slon', 'ptak', 'ryba']
dlugie = list(filter(lambda s: len(s) > 3, slova))
print(dlugie)
```

Funkcja reduce

Funkcja reduce (z modułu functools) **redukuje iterowalny do pojedynczej wartości, stosując funkcję kumulacyjną do elementów**. Wymaga importu: **from functools import reduce**.

Ogólna składnia

```
reduce(function, iterable)
```

- **function** $\rightarrow$ funkcja z dwoma argumentami
- **iterable** $\rightarrow$ lista / tuple / inna kolekcja

Python bierze elementy **po kolei i łączy je w jeden wynik**.

Przykład zastosowania: Obliczenie sumy elementów listy.

```
from functools import reduce
liczby = [1, 2, 3, 4]
suma = reduce(lambda x, y: x + y, liczby)
print(suma)
```

Przykład: Znalezienie maksymalnej wartości w liście.

```
from functools import reduce
liczby = [1, 3, 2, 5, 4]
maks = reduce(lambda x, y: x if x > y else y, liczby)
print(maks)
```

Różnica między map, filter, reduce

Funkcja	Co robi
map	zmienia każdy element
filter	wybiera elementy
reduce	redukuje wszystko do jednej wartości

Dekoratory

Dekoratory **to funkcje, które modyfikują lub rozszerzają zachowanie innych funkcji bez zmiany ich kodu**. Definiuje się je z użyciem **@dekorator nad funkcją**. Są meta-funkcjami, które zwracają nową funkcję.

Idea dekoratora

Dekorator:

- bierze **funkcję jako argument**
- **opakowuje ją dodatkową logiką**
- zwraca **nową funkcję**

Czyli zamiast zmieniać funkcję, **dodajemy warstwę wokół niej**.

Przykład

```
def moj_dekorator(func):
    def wrapper():
        print("Coś przed funkcją")
        func()
        print("Coś po funkcji")
    return wrapper
```

```
@moj_dekorator
def powitanie():
    print("Witaj!")
```

```
powitanie()
```

Przykład zastosowania: Dekorator mierzący czas wykonania funkcji.

```
import time
```

```
def miernik_czasu(func):
    def wrapper(*args, **kwargs):
```

```

    start = time.time()
    wynik = func(*args, **kwargs)
    koniec = time.time()
    print(f"Czas wykonania: {koniec - start} sekund")
    return wynik
return wrapper

@miernik_czasu
def wolna_funkcja(n):
    time.sleep(n)
    return "Gotowe"

print(wolna_funkcja(2))

```

Przykład: Dekorator dodający logowanie.

```

def loguj(func):
    def wrapper(*args, **kwargs):
        print(f"Wywołano {func.__name__} z argumentami {args}")
        return func(*args, **kwargs)
    return wrapper

@loguj
def dodaj(a, b):
    return a + b

print(dodaj(3, 5))

```

Lekcja - 1 kwietnia sprawdzian

Temat: is oraz ==, funkcja rekurencyjna. Typy mutowalne i niemutowalne, inkrementacja, dekrementacja. Try-except-else-finally, raise i custom exceptions

Operator == porównuje wartość. Inaczej mówiąc **sprawdza, czy wartości dwóch obiektów są takie same. Nie bierze pod uwagę, czy obiekty są przechowywane w tym samym miejscu w pamięci – liczy się tylko zawartość.** Jest to porównanie semantyczne, oparte na metodzie `__eq__` obiektu.

Przykłady

```

a = 5
b = 5
print(a == b)

```

```

a = [1, 2, 3]
b = [1, 2, 3]

```

```
print(a == b)
```

Operator **is** porównanie obiektu w pamięci. Inaczej mówiąc sprawdza, czy dwie zmienne wskazują na ten sam obiekt w pamięci.

Przykłady

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is b)
```

```
a = [1, 2, 3]
b = a
print(a is b)
```

Python ma mechanizm zwany **internowaniem małych liczb całkowitych** (integer caching).

- Dla liczb z zakresu **-5 do 256** Python **przechowuje jedną wspólną instancję**
- Czyli np. liczba 10 istnieje tylko raz w pamięci

```
a = 10
b = int("10")
print(a is b) # True
```

```
c = 256 # -5 do 256
d = int("256")
print(c is d) # True
```

```
e = 1000
f = int("1000")
print(e is f) # False
```

Przykład – list

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
print(a == b)
print(a is b)
```

Przykład – dict

```
a = {"name": "Jan"}
b = {"name": "Jan"}
```

```
print(a == b)
print(a is b)
```

Przykład – set

```
a = {1, 2, 3}
b = {1, 2, 3}
```

```
print(a == b)
print(a is b)
```

Przykład – is – None

```
x = None
```

```
if x is None:
    print("brak wartości")
```

Rekurencja to technika programowania, w której **funkcja wywołuje samą siebie, aby rozwiązać problem. Zamiast używać pętli** (jak `for` czy `while`), **funkcja dzieli problem na mniejsze podproblemy**, aż dojdzie do prostego przypadku, który można rozwiązać bezpośrednio. Kluczowe elementy funkcji rekurencyjnej:

- **Przypadek bazowy (base case):** Warunek, który kończy rekurencję. Bez niego funkcja będzie się wywoływać w nieskończoność, co spowoduje błąd (`RecursionError` w Pythonie, bo przekroczony zostanie limit rekurencji, domyślnie ok. 1000 wywołań).
- **Przypadek rekurencyjny (recursive case):** Część, w której funkcja wywołuje siebie z mniejszymi argumentami, zbliżając się do przypadku bazowego.

Rekurencja jest przydatna w problemach, które mają strukturę drzewiastą lub dzielą się na podproblemy, np. obliczanie silni, przeszukiwanie drzew, sortowanie (jak `quicksort`) czy generowanie fraktali.

Przykład: Obliczanie silni

```
def silnia(n):  
    if n == 0 or n == 1: # Przypadek bazowy: 0! = 1, 1! = 1  
        return 1  
    else: # Przypadek rekurencyjny  
        return n * silnia(n - 1)
```

Jak to działa?

- Dla `silnia(5)`: Wywołuje `5 * silnia(4)`
- `silnia(4)`: Wywołuje `4 * silnia(3)`
- `silnia(3)`: Wywołuje `3 * silnia(2)`
- `silnia(2)`: Wywołuje `2 * silnia(1)`
- `silnia(1)`: Zwraca `1` (przypadek bazowy)
- Teraz "wspinamy się" z powrotem: $2 * 1 = 2$, $3 * 2 = 6$, $4 * 6 = 24$, $5 * 24 = 120$.

Typy mutowalne i niemutowalne

W Pythonie typy danych dzielą się na **mutowalne (mutable)** i **niemutowalne (immutable)** w zależności od tego, czy można je modyfikować po utworzeniu. To kluczowa koncepcja, bo wpływa na to, jak działają przypisania, funkcje i struktury danych (np. listy jako klucze w słownikach nie działają, bo są mutowalne).

- **Niemutowalne (immutable):** Po stworzeniu obiektu nie można zmienić jego wartości. Zamiast modyfikować, Python tworzy nowy obiekt. To bezpieczniejsze w kontekstach wielowątkowych i jako klucze w słownikach (muszą być hashable).
- **Mutowalne (mutable):** Można zmieniać zawartość obiektu bezpośrednio, bez tworzenia nowego. To przydatne dla dynamicznych struktur, ale może prowadzić do nieoczekiwanych efektów (np. modyfikacja listy w funkcji wpływa na oryginalną listę).

Użyję tabeli do porównania powszechnych typów:

Typ danych	Mutowalny?	Przykłady użycia	Uwagi
int (liczba całkowita)	Nie	x = 5; x = 6 (tworzy nowy obiekt)	Zmiana wartości to nowe przypisanie.
float (liczba zmiennoprzecinkowa)	Nie	y = 3.14	Jak int, niezmiennialne.
str (ciąg znaków)	Nie	s = "hello"; s.upper() (zwraca nowy string)	Metody jak replace() tworzą nowy obiekt.
tuple (krotka)	Nie	t = (1, 2, 3)	Nie można dodać/usunąć elementów.
frozenset (zamrożony zbiór)	Nie	fs = frozenset([1, 2])	Jak set, ale niezmiennialny.
list (lista)	Tak	l = [1, 2]; l.append(3) (modyfikuje oryginalną)	Można dodawać, usuwać, sortować.
dict (słownik)	Tak	d = {'a': 1}; d['b'] = 2	Dodawanie/usuwanie kluczy/wartości.
set (zbiór)	Tak	s = {1, 2}; s.add(3)	Dynamiczne dodawanie/usuwanie.
bytearray (tablica bajtów)	Tak	ba = bytearray(b'abc'); ba[0] = 65	Modyfikowalna wersja bytes.

Różnice w praktyce

- **Przypisanie i modyfikacja:** Dla typów niemutowalnych, jak string, s = "hello"; s = s + " world" tworzy nowy string. Dla list: l = [1]; l += [2] modyfikuje istniejącą listę.
- **Funkcje i argumenty:** Jeśli przekażesz mutowalny obiekt do funkcji, zmiany w funkcji wpłyną na oryginalny obiekt (przekazywanie przez referencję). Dla niemutowalnych – nie.
- **Hashowalność:** Tylko niemutowalne typy mogą być kluczami w słownikach lub elementami w setach, bo ich hash nie zmienia się.

Przykład kodu

```
# Niemutowalny: string
s = "hello"
s2 = s # s2 wskazuje na ten sam obiekt
s = s.upper() # Tworzy nowy obiekt, s2 zostaje "hello"
print(s) # "HELLO"
print(s2) # "hello"

# Mutowalny: lista
l = [1, 2]
l2 = l # l2 wskazuje na tę samą listę
l.append(3) # Modyfikuje oryginalną listę
print(l) # [1, 2, 3]
print(l2) # [1, 2, 3] - też zmienione!
```

Python nie ma ++ i --

Python jest zaprojektowany, aby być prostym i czytelnym (zgodnie z "Zen of Python" – "Readability counts"). Operatory ++ i -- są skrótami, które mogą być mylące, bo działają różnie w zależności od kontekstu (np. pre-inkrementacja ++x vs. post-inkrementacja x++). **W Pythonie preferuje się jawne operacje, jak `x = x + 1` lub `x += 1`, co jest łatwiejsze do zrozumienia na pierwszy rzut oka.**

```
x = 5
x++ # Błąd: SyntaxError: invalid syntax
```

W JavaScript np.:

Pre-inkrementacja/dekrementacja: Najpierw modyfikuje zmienną, potem zwraca nową wartość.

- ++x: Zwiększ x o 1, zwróć nowe x.
- --x: Zmniejsz x o 1, zwróć nowe x.

Post-inkrementacja/dekrementacja: Najpierw zwraca bieżącą wartość, potem modyfikuje zmienną.

- x++: Zwróć bieżące x, potem zwiększ o 1.
- x--: Zwróć bieżące x, potem zmniejsz o 1.

Python **nie ma wbudowanych operatorów ++ ani --**. Zamiast tego używa się operatorów **przypisania złożonego: += dla inkrementacji i -= dla dekrementacji**. To proste i czytelne, ale wymaga jawnego zapisu. Te operatory działają na zmiennych liczbowych (int, float), ale też na niektórych kolekcjach (np. listy z += do dodawania elementów).

Podstawowe przykłady

```
# Inkrementacja
x = 5
x += 1 # To samo co x = x + 1
print(x) # Wyjście: 6

# Dekrementacja
y = 10
y -= 2 # To samo co y = y - 2
print(y) # Wyjście: 8

# Można używać z innymi wartościami
z = 3.5
z += 0.5 # Inkrementacja o 0.5
print(z) # Wyjście: 4.0

# Dla stringów lub list (rozszerzone użycie +=)
s = "Witaj"
s += " świecie!" # Konkatenacja
print(s) # Wyjście: Witaj świecie!

lista = [1, 2]
lista += [3] # Dodawanie elementu
print(lista) # Wyjście: [1, 2, 3]
```

Inkrementacja i dekrementacja często pojawiają się w pętlach, aby kontrolować liczniki lub iterować wstecz. Python preferuje pętle **for z range()**, które automatycznie obsługują inkrementację, ale w **while** musisz to robić ręcznie.

1. Pętla for z inkrementacją (używając range):

- a. `range(start, stop, step)` automatycznie inkrementuje (`step=1` domyślnie) lub dekrementuje (`step=-1`).

```
# Inkrementacja w pętli for (od 0 do 4)
for i in range(5): # i zaczyna od 0, inkrementuje o 1 co iterację
    print(i) # Wyjście: 0 1 2 3 4

# Dekrementacja w pętli for (od 5 do 1)
for j in range(5, 0, -1): # Start=5, stop=0 (nie inkluzyny), step=-1
    print(j) # Wyjście: 5 4 3 2 1

# Ręczna inkrementacja w pętli for (rzadziej używana, ale możliwe)
licznik = 0
for _ in range(3): # Pętla 3 razy
    licznik += 2 # Inkrementacja o 2
    print(licznik) # Wyjście: 2 4 6
```

Pętla while z inkrementacją/dekrementacją:

- Tutaj operatory `+=` i `-=` są kluczowe, bo kontrolujesz warunek ręcznie.

```
# Inkrementacja w while (licznik rosnący)
licznik = 0
while licznik < 5:
    print(licznik) # Wyjście: 0 1 2 3 4
    licznik += 1 # Inkrementacja

# Dekrementacja w while (licznik malejący)
licznik = 10
while licznik > 0:
    print(licznik) # Wyjście: 10 9 8 ... 1
    licznik -= 1 # Dekrementacja

# Przykład z warunkiem i inkrementacją o zmienną wartość
suma = 0
i = 1
while suma < 20:
    suma += i # Inkrementacja o i (1, potem 2, itd.)
    i += 1 # Inkrementacja i
    print(suma) # Wyjście: 1 3 6 10 15 21 (zatrzyma się przed 21, bo warunek)
```

Wyjątki w Pythonie: Try-except-else-finally, raise i custom exceptions

Wyjątki to mechanizm, który pozwala programowi radzić sobie z błędami w czasie wykonania (**runtime errors**), np. dzielenie przez zero, brak pliku czy nieoczekiwane dane. Zamiast **crashować program**, możesz "złapać" błąd i obsłużyć go elegancko. To kluczowe dla stabilnego kodu. Python ma wbudowane wyjątki (np. **ZeroDivisionError**, **FileNotFoundError**), ale możesz też tworzyć własne. Podstawowa struktura to blok try-except, z opcjonalnymi else i finally.

Podstawowa struktura: try-except

- **try:** W tym bloku umieszczasz kod, który może spowodować błąd.
- **except:** Łapie wyjątek i obsługuje go. Możesz sprecyzować typ wyjątku (np. `except ValueError:`) lub złapać wszystkie (`except:` – ale to niezalecane, bo maskuje błędy).

Przykład

```
try:
    liczba = int(input("Podaj liczbę: ")) # Może rzucić ValueError, jeśli nie liczba
    wynik = 10 / liczba # Może rzucić ZeroDivisionError
    print(wynik)
except ValueError:
    print("To nie jest liczba!")
except ZeroDivisionError:
    print("Nie dziel przez zero!")
```

Dodatkowe bloki: else i finally

- **else:** Wykonuje się tylko, jeśli w try NIE wystąpił żaden wyjątek. Przydatne do kodu, który ma działać po sukcesie.
- **finally:** Zawsze się wykonuje, niezależnie od tego, czy był wyjątek czy nie. Idealne do czyszczenia zasobów (np. zamykanie plików).

```
try:
    a = int(input("Podaj pierwszą liczbę: ")) # Może rzucić ValueError
    b = int(input("Podaj drugą liczbę: ")) # Może rzucić ValueError
    wynik = a / b # Może rzucić ZeroDivisionError
except ValueError:
    print("Jedna z wartości nie jest liczbą całkowitą!")
except ZeroDivisionError:
    print("Nie można dzielić przez zero!")
except Exception as e:
    print(f"Nieoczekiwany błąd: {e}")
else:
    print(f"Wynik dzielenia: {wynik}")
    print("Obliczenia zakończone sukcesem.")
finally:
    print("Program zakończył przetwarzanie wejścia – zawsze to się wyświetli.")
```

Raise: Rzucanie wyjątków `raise` pozwala ręcznie "rzucić" wyjątek, np. gdy chcesz przerwać wykonanie przy niepoprawnych danych. Możesz `raise`'ować wbudowany wyjątek lub własny.

Przykład:

```
def sprawdz_wiek(wiek):
    if wiek < 18:
        raise ValueError("Jesteś za młody!") # Rzuca wyjątek z komunikatem
    return "OK"

try:
    sprawdz_wiek(15)
```

```
except ValueError as e:  
    print(e) # Wyjście: Jesteś za młody!
```

Custom exceptions: Własne wyjątki Możesz tworzyć własne klasy wyjątków, dziedzicząc po Exception (lub podklasach jak ValueError). To przydatne w dużych projektach, by mieć specyficzne błędy. Przykład tworzenia i użycia:

```
class MojBład(Exception): # Dziedziczy po Exception  
    def __init__(self, wiadomosc="To mój custom błąd!"):   
        self.wiadomosc = wiadomosc  
        super().__init__(self.wiadomosc)  
  
def funkcja():  
    raise MojBład("Coś poszło nie tak.")  
  
try:  
    funkcja()  
except MojBład as e:  
    print(e) # Wyjście: To mój custom błąd!
```

Lekcja 9

Temat: Sprawdzian wiadomości